

Application Note

RFID Tag Write Behavior in Current Build

Document ID: AN-RFID-WRITE-CURRENT-BEHAVIOR

Date: 2026-05-21

Applies To: TC22R, EM45, eConnex, Zebra RFID SDK (API3)

1. Purpose

This note documents the exact current write behavior implemented in the app, with publication-ready snippets taken from the active code path.

2. Current Behavior Summary

1. The write dialog button is labeled "Write with TID Filter".
2. The active write path in `writeWithTidPrefilter(...)` currently calls direct write.
3. TID prefilter code exists in the project but is not active in the current `writeWithTidPrefilter(...)` execution path.
4. `RFIDHandler.read(...)` supports inventory-filter-aware access via `readWait(..., true)`.
5. `RFIDHandler.write(...)` supports null target ID by converting empty input to `effectiveTagID = null`.

3. MainActivity Active Snippets

3.1 TID normalization helper

```
```kotlin
private fun buildTidFilterHex(rawTid: String): String {
 val hexOnly = rawTid
 .replace(" ", "")
 .uppercase(Locale.US)
 .filter { it in "0123456789ABCDEF" }

 // 32 hex chars = 128 bits
 return if (hexOnly.length > 32) hexOnly.substring(0, 32) else hexOnly
}
```
```

3.2 Active write entry

```
```kotlin
private fun writeWithTidPrefilter(
 targetTagId: String,
 data: String,
 selectedBank: MEMORY_BANK,
 offset: Int,
 tidWordCount: Int = 8
) {
 val handler = rfidHandler
 if (handler == null) {
 sendToast("RFID handler unavailable")
 return
 }

 if (targetTagId.isBlank()) {
 sendToast("Tag ID is empty")
 return
 }

 if (data.isBlank() || data.length % 4 != 0) {

```

```

 sendToast("Write data must be hex and length multiple of 4")
 return
 }

 handler.write(targetTagId, data, selectedBank, offset)
}...

4. RFIDHandler Access Snippets

4.1 Apply TID prefilter (available in code)

```kotlin
public fun applyTagFilter(tag: String) {
    val currentReader = reader
    if (currentReader == null) {
        Log.e(TAG, "RFIDReader is null")
        return
    }
    //E280119120007676B2EF034A000000009000

    // Add state aware pre-filter
    val filters = Prefilters()
    // In Kotlin, inner classes are instantiated by calling the constructor on the
    // outer instance
    val filter = filters.Prefilter()

    filter.setAntennaID(1.toShort()) // Explicitly cast to Short in Kotlin
    filter.setTagPattern(tag)
    if(tag.length*4 > 128)
        filter.setTagPatternBitCount(128)
    else
        filter.setTagPatternBitCount(tag.length * 4)

    filter.setBitOffset(0) // skip PC bits (always it should be in bit length)
    filter.setMemoryBank(MEMORY_BANK.MEMORY_BANK_TID)
    filter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE)

    // inventoried flag of session S1 of matching tags to B
    filter.StateAwareAction.setTarget(TARGET.TARGET_INVENTORIED_STATE_S0)

    // not to select tags that match the criteria
    filter.StateAwareAction.setStateAwareAction(STATE_AWARE_ACTION.STATE_AWARE_ACTION_INV_B_NOT_INV_A)

    try {
        // Clear existing pre-filters before adding new ones
        currentReader.Actions.Prefilters.deleteAll()

        // Add the new filter
        currentReader.Actions.Prefilters.add(filter)
        Log.i(TAG, "State-aware pre-filter added successfully for tag pattern: $tag")
    } catch (e: InvalidUsageException) {
        Log.e(TAG, "InvalidUsageException: ${e.vendorMessage}")
        e.printStackTrace()
    } catch (e: OperationFailureException) {
        Log.e(TAG, "OperationFailureException: ${e.vendorMessage} Status: ${e.results}")
        e.printStackTrace()
    }
}

```

```
}...
```

4.2 Set singulation (available in code)

```
```kotlin
public fun setSingulationForFilter(bSet: Boolean) {
 try {
 // Use safe call ?. to ensure reader is not null
 val s0SingulationControl = reader?.Config?.Antennas?.getSingulationContr
ol(1) ?: return

 // Kotlin property access for setters
 s0SingulationControl.session = SESSION.SESSION_S0

 // Accessing the Action nested object
 if(bSet)
 s0SingulationControl.Action.inventoryState = INVENTORY_STATE.INVENTO
RY_STATE_B
 else
 s0SingulationControl.Action.inventoryState = INVENTORY_STATE.INVENTO
RY_STATE_A

 s0SingulationControl.Action.setPerformStateAwareSingulationAction(true)
 // Apply the configuration back to the reader
 reader?.Config?.Antennas?.setSingulationControl(1, s0SingulationControl)

 } catch (e: InvalidUsageException) {
 e.printStackTrace()
 } catch (e: OperationFailureException) {
 e.printStackTrace()
 }
}
}...
```

### ### 4.3 Write implementation (active)

```
```kotlin
public fun write(selectTagID: String, dataToWrite: String, bank: MEMORY_BANK, wo
rdOffset: Int) {
    if (executor.isShutdown) {
        executor = Executors.newSingleThreadExecutor()
    }
    executor.execute {
        if (dataToWrite.isEmpty() || dataToWrite.length % 4 != 0) {
            context?.sendToast("Write Failed: Invalid data length")
            return@execute
        }
        try {
            if (isReaderConnected()) {
                reader?.Actions?.Inventory?.stop()
                Thread.sleep(100)
            }
        } catch (e: Exception) {
            Log.e(TAG, "Stop inventory failed", e)
        }

        val tagData = TagData()
        val tagAccess = TagAccess()
        val writeAccessParams = tagAccess.WriteAccessParams()

        writeAccessParams.apply {
```

D

ams, null,

r0fWords)

Words)

}

```
```kotlin
```

```

public fun read(selectTagID: String, bank: MEMORY_BANK, wordOffset: Int, wordCount: Int, callback: (String?) -> Unit) {
 if (executor.isShutdown) {
 executor = Executors.newSingleThreadExecutor()
 }
 executor.execute {
 try {
 if (isReaderConnected()) {
 reader?.Actions?.Inventory?.stop()
 Thread.sleep(100)
 }
 } catch (e: Exception) {
 Log.e(TAG, "Stop inventory failed", e)
 }
 }

 val tagAccess = TagAccess()
 val readAccessParams = tagAccess.ReadAccessParams()

 readAccessParams.apply {
 accessPassword = 0
 memoryBank = bank
 offset = wordOffset
 count = wordCount
 }

 try {

```

